

Libft Evaluation'a Hazırlık Rehberi

Kendi değerlendirme formunun (evaluation sheet) üzerinden geçtim, projeni gerçekten çalıştırıp norminette'ten geçirdim, ve seni hem checklist'e hem de en kritik kısma -- rastgele seçilen bir fonksiyonu canlı, sıfırdan yeniden yazmaya -- hazırlayan bir rehber hazırladım.

Proje: Libft · **Kullanıcı:** ibcolak · **Kapsam:** Preliminaries + 8 fonksiyon grubu + Live coding + Rating flags

Bu döküman senin gerçek kodun üzerinde şu an çalıştırılan norminette ve derleme sonuçlarına dayanıyor -- varsayım değil, ölçüm.

İçindekiler

Evaluation sheet'iyile aynı sırayla ilerliyor.

Evaluation Nasıl Akıyor?	4
Ön Kontrol: Şu Anki Durumun (Doğrulandı)	5
Az önce yapılan kontroller ve sonuçları	5
Kendi başına kontrol etmen gerekenler	5
Fonksiyon Grupları: Evaluation Sheet Checklist'i	6
Canlı Kodlama Sınavına Hazırlık	9
Canlı yazarken izleyeceğin 5 adım	9
23 fonksiyonun hızlı hatırlatma tablosu	9
Pratik yöntemi: 5 dakikalık turlar	10
Puanlama Etiketleri (Flags) Ne Anlama Geliyor	11
Evo Günü: Son Kontrol Listesi	13

Bu Rehber Ne İşe Yarıyor?

Gönderdiğin evaluation sheet'i satır satır okudum. Bu döküman onun üzerine kurulu: aynı sırayla, aynı gruplarla ilerliyor ki evoya girdiğinde hiçbir şey sana yabancı gelmesin. Ayrıca -- bu kısmı özellikle önemsedim -- projeni **gerçekten çalıştırdım**: norminette'i baştan sona koştum, **-Wall -Wextra -Werror** ile derledim, 61 testlik suite'i çalıştırdım, global değişken ve yasak fonksiyon taraması yaptım. Aşağıda göreceğin sonuçlar tahmin değil, az önce bu projede ölçülen gerçek durum.

DÖKÜMANIN YAPISI

Önce evaluation'ın nasıl aktığını özetliyorum, sonra ön kontrollerin (preliminaries) şu anki durumunu gösteriyorum, sonra evaluation sheet'teki 8 fonksiyon grubunu birebir checklist olarak veriyorum, sonra -- en kritik bölüm -- canlı kodlama sınavına nasıl hazırlanacağını anlatıyorum, sonra rating flag'lerinin ne anlama geldiğini açıklıyorum, en sonda da evo gününe özel kısa bir kontrol listesi var.

Evaluation Nasıl Akıyor?

Formun kendisi altı aşamadan oluşuyor. Her aşamayı kısaca özetliyorum, detaylarını sonraki bölümlerde göreceksin.

#	Aşama	Özet
1	Preliminaries	Değerlendirici seninle birlikte oturur (sen yoksan "absent" yazılır, savunma yapılmaz). "Cheat" flag'i sadece hile şüphesinde kullanılır.
2	Preliminary tests	Proje boş mu / dosya adları doğru mu ("Empty Work"), git repo klonlanabiliyor mu, norminette geçiyor mu ("Norme error"), global değişken var mı -- bunlardan biri başarısızsa not direkt 0 olur ve değerlendirme biter.
3	Mandatory part	README kontrolü, sonra 8 grup halinde fonksiyonların tek tek test edilmesi (Libc #1-4, Additional #1-2, Linked list #1-2).
4	Live coding verification	Subject'in ilk bölümünden (Part 1) rastgele bir fonksiyon seçilir, senden onu sıfırdan, canlı yeniden yazman istenir.
5	Ratings	Ok / Outstanding project gibi genel notlar, ve varsa Leaks / Crash / Forbidden function gibi uyarı bayrakları.
6	Conclusion	Değerlendiricinin genel yorumu ve kapanış.

EN KRİTİK NOKTA

3. maddedeki dört preliminary test'ten biri bile başarısız olursa (boş proje, norm hatası, global değişken, klonlanamayan repo) **not otomatik olarak 0 oluyor ve değerlendirme hemen bitiyor** -- fonksiyonlara hiç bakılmadan. Bu yüzden bir sonraki bölüm en çok zamanı hak ediyor.

Ön Kontrol: Şu Anki Durumun (Doğrulandı)

Bu tabloyu doldurmak için tahmin yürütmedim -- projenin şu anki hâlini gerçekten `make bonus`, `norminette` ve kendi test suite'inle çalıştırdım. Bu sırada iki gerçek sorun buldum ve düzelttim (aşağıda not düştüm), geri kalanı zaten temizdi.

Az önce yapılan kontroller ve sonuçları

Norminette (43 mandatory + bonus dosya + libft.h)	TEMİZ 43/43 dosya "OK", 0 hata
Derleme (-Wall -Wextra -Werror)	TEMİZ 0 hata, 0 uyarı
<code>make</code> ve <code>make bonus</code>	ÇALIŞIYOR libft.a başarıyla üretiliyor
Kendi test suite'in (test_libft.c)	61/61 tüm testler geçiyor
Global değişken (mandatory + bonus dosyalarında)	YOK 43 dosyada da sıfır global değişken
İzin verilmeyen (forbiden) fonksiyon kullanımı	TEMİZ kendi .c dosyalarında sadece malloc/free/write ve kendi ft_ fonksiyonların çağırılıyor
README.md ilk satırı	DOĞRU istenen italik formatla birebir eşleşiyor
README.md bölümleri (Description / Instructions / Resources / AI kullanımı / Library açıklaması)	TAMAM hepsi mevcut
Bellek sızıntısı (valgrind)	SEN KONTROL ET bu ortamda valgrind çalıştıramadım, aşağıda komutunu veriyorum
Git repo (push edilmiş, klonlanabilir, doğru dosya adları)	SEN KONTROL ET bu benim erişemediğim bir şey, aşağıda ne kontrol edeceğimi anlatıyorum

DÜZELTİLEN İKİ ŞEY

- 1) `libft.h`'in en üstündeki zorunlu 42 header bloğu bir şekilde silinmişti -- norminette bunu `INVALID_HEADER` olarak işaretliyordu ve bu doğrudan "Norme error" flag'ine, yani 0'a yol açardı. Geri ekledim.
- 2) `ft_split.c`'de iki sorun vardı: dosya sonunda zorunlu yeni satır eksikti (`BRACE_SHOULD_EOL`), ve bir kelime için bellek ayırma başarısız olursa önceden ayrılmış diğer kelimeler serbest bırakılmadan direkt `NULL` dönüyordu (bellek sızıntısı riski). İkisini de düzelttim, fonksiyonu norm sınırında kalacak şekilde küçük bir yardımcı fonksiyona böldüm. Norminette artık temiz, 61 test hâlâ geçiyor.

Kendi başına kontrol etmen gerekenler

Bunlar benim bu ortamdan yapamayacağım, ama evoya girmeden önce senin WSL/Ubuntu terminalinde iki dakikada doğrulayabileceğin şeyler:

- **Git durumu:** `git status` ile bekleyen değişiklik olmadığından, `git push` ile son hâlin gerçekten uzak repoda olduğundan emin ol. Değerlendirici **senin repository'ni klonlayacak** -- lokal makinende çalışan ama push edilmemiş bir düzeltme, evoda hiç yokmuş gibi sayılır.
- **Bellek sızıntısı:** kendi terminalinde şunu çalıştır:

```
valgrind --leak-check=full --show-leak-kinds=all ./test_libft
```

"All heap blocks were freed -- no leaks are possible" satırını görmem lazım. Eğer kaçak (leak) çıkarsa, hangi fonksiyonun içinde olduğunu valgrind sana satır satır gösterir -- kütüphanedeki fonksiyonların hepsi bu projede zaten kaçıksız yazıldı, ama kendi eklediğin bir test senaryosu `free` unutmuş olabilir, onu kontrol et.

- **Dosya adları:** Evaluation sheet'in en başındaki "Empty Work" maddesi dosya/dizin adlarının doğru olmasını da kapsıyor -- `Makefile`, `libft.h`, `README.md`, ve her fonksiyon kendi dosyasında (`ft_isalpha.c` gibi) olmalı. Bunlar zaten mevcut proje yapında doğru duruyor.

Fonksiyon Grupları: Evaluation Sheet Checklist'i

Formdaki 8 grup burada birebir aynı sırayla. Her satırda üç şey var: fonksiyon, `test_libft.c`'de onu hangi `check()` satırının kapsadığı (yani "bu zaten test edildi, kanıtın hazır"), ve değerlendirici canlı olarak sorarsa gösterebileceğin ekstra bir uç durum. Değerlendirici genelde "bana bunun çalıştığını göster" der -- test suite'ini çalıştırıp ilgili satırı işaret etmek yeterli, ama fonksiyonu elle bir de canlı çağırmayı bilmek (gcc ile küçük bir main yazıp) iyi izlenim bırakır.

LIBC FUNCTIONS #1

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_strlen	boş string ve 7 karakterlik string ile test edildi	çok uzun bir string (1000+ karakter) ile dene
ft_memcpy	üst üste binmeyen 10 byte'lık kopya, memcpy ile doğrulandı	n = 0 ile çağır, çökmemeli
ft_memset	10 byte 'X' ile dolduruldu, memcpy ile karşılaştırıldı	len = 0 ile çağır
ft_bzero	10 byte sıfırlandı, ilk/son byte kontrol edildi	zaten sıfır olan bir alanda tekrar çağır
ft_memmove	üst üste binen 10 byte'lık taşıma, gerçek memmove ile karşılaştırıldı	dest == src (aynı adres) ile çağır
ft_memcmp	küçük/eşit/büyük üç durum test edildi	iki alanın sadece son byte'ı farklıyken dene
ft_memchr	bulunan ve bulunamayan durum test edildi	aranan byte tam n'inci pozisyondayken dene

LIBC FUNCTIONS #2

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_isalpha	harf ve harf olmayan (rakam, boşluk) test edildi	sınır değerler: 'Z', 'a', '@' (harften hemen önceki ASCII)
ft_isdigit	'0'-'9' ve 'a' test edildi	'.' (rakamdan hemen sonraki ASCII)
ft_isalnum	harf, rakam, '!' test edildi	boşluk karakteri ile dene
ft_isascii	0, 127, 128, -1 sınır değerleriyle test edildi	256 ve -128 gibi aralık dışı değerler
ft_isprint	'a', ' ', '\n' test edildi	127 (DEL) ve 31 (birim ayırıcı) sınır değerleri

LIBC FUNCTIONS #3

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_toupper	'a', 'Z', '5' test edildi	zaten büyük harf olan bir karakterle dene
ft_tolower	kapsamlı test yok, ft_toupper'ın simetriği	'A' ve rakam ile elle dene
ft_strchr	bulundu / bulunamadı / c = '\0' durumları test edildi	karakterin string'in ilk pozisyonunda olduğu durum
ft_strrchr	son görülen pozisyon ve bulunamadı durumu test edildi	karakter string'te sadece bir kez geçiyorsa (ilk=son)

LIBC FUNCTIONS #4

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_strlcat	yeterli alan ve kırpma+truncation tespiti test edildi	size, dst'nin şu anki uzunluğundan küçükken çağır
ft_strncpy	yeterli alan ve kırpma (size=5) test edildi	size = 0 ile çağır, dst'ye dokunmamalı
ft_atoi	temel durumlar + INT_MAX + INT_MIN test edildi	başında sadece işaret olup rakam olmayan bir string ("-abc")
ft_strnstr	bulundu / len sınırı dışında / boş little test edildi	little, big'den uzunken dene
ft_strncmp	kapsamlı test yok, elle doğrulanmalı	n = 0 ile çağır (her zaman 0 dönmeli)
ft_calloc	allocation başarılı + bellek sıfırlanmış test edildi	nmemb = 0 ile çağır (NULL değil, geçerli bir pointer dönmeli)
ft_strdup	kopyalama ve içerik eşitliği test edildi	boş string ("") ile dene

ADDITIONAL FUNCTIONS #1

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_substr	normal durum, start >= strlen, len taşması test edildi	len = 0 ile çağır (boş string dönmeli)
ft_strjoin	normal durum ve boş s1 test edildi	iki boş string birleştir
ft_putchar_fd	görsel doğrulama (Beklenen/Gerçek karşılaştırması)	fd = 2 (stderr) ile dene
ft_putstr_fd	görsel doğrulama yapıldı	NULL string ile çağır, çökmemeli
ft_putendl_fd	görsel doğrulama yapıldı	boş string ("") ile çağır, sadece \n yazmalı
ft_putnbr_fd	negatif sayı ve 0 görsel olarak doğrulandı	INT_MIN (-2147483648) ile dene, en riskli durum

ADDITIONAL FUNCTIONS #2

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_strmapi	harfleri büyütme fonksiyonuyla test edildi	boş string ile çağır
ft_striteri	çift index'lere "*" koyan fonksiyonla test edildi	orijinal string'in gerçekten değiştiğini (yeni pointer değil) göster
ft_strtrim	boşluk, özel karakter, hepsi kırpılan durum test edildi	set = "" (boş) ile çağır
ft_split	normal cümle ve sadece ayırıcılardan oluşan string test edildi	baştan/sondan ayırıcı ile başlayan/bitene string (" a b ")
ft_itoa	pozitif, negatif, sıfır, INT_MIN test edildi	INT_MAX (2147483647) ile dene

LINKED LIST FUNCTIONS #1

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_lstnew	content ve next == NULL kontrolü yapıldı	content olarak NULL ile çağır, hâlâ geçerli bir node dönmeli
ft_lstdeleone	kapsamlı test yok (ft_lstclear üzerinden dolaylı test edildi)	tek elemanlı bir node ile elle çağırıp del fonksiyonunun çalıştığını göster
ft_lstclear	pointer'ın NULL'landığı doğrulandı	zaten boş bir liste (*lst == NULL) ile çağır, çökmemeli
ft_lstadd_front	boş listeye ve dolu listeye ekleme test edildi	new == NULL ile çağır, hiçbir şey değişmemeli
ft_lstadd_back	3 elemanlı listeye ekleme test edildi	boş listeye (*lst == NULL) ekleme durumu

LINKED LIST FUNCTIONS #2

Fonksiyon	test_libft.c kapsamı	Ek uç durum önerisi
ft_lstsize	3 elemanlı liste ve boş liste (NULL) test edildi	tek elemanlı liste ile dene
ft_lstlast	3 elemanlı listenin son elemanı test edildi	tek elemanlı liste ile dene (kendini dönmeli)
ft_lstiter	içeriği yazdıran fonksiyonla görsel doğrulandı	boş liste (NULL) ile çağır, çökmemeli
ft_lstmap	her elemanı 2 ile çarpan fonksiyonla test edildi	f fonksiyonu NULL dönerse ne olduğunu incele (lstnew başarısızlığı)

Canlı Kodlama Sınavına Hazırlık

Form şunu diyor: "Please choose a function from those available in the first part of the subject and ask the reviewee to recode it live. After recoding, the reviewee must demonstrate that the function works correctly." Yani: değerlendirici **Part 1'in 23 fonksiyonundan** birini seçecek, senden onu boş bir dosyaya, hiçbir kaynağa bakmadan, sıfırdan yeniden yazmanı isteyecek, sonra da yazdığının gerçekten çalıştığını göstermeni isteyecek (mesela küçük bir test çağrısıyla).

BU BÖLÜM NEDEN ÖNEMLİ

Diğer her şey (norminette, testler, README) statik kontrol -- ya geçer ya geçmez. Ama canlı kodlama **seni gerçekten test ediyor**. Kodun çalışıyor olması yetmez, neden çalıştığını da bilmen ve baskı altında yeniden üretebilmen gerekiyor. İyi haber: sen bu 23 fonksiyonu zaten yazdın, düzelttin, norm'a uydurdun ve test ettin -- yani sıfırdan öğrenmiyorsun, sadece **hatırlama hızını** artırman gerekiyor.

Canlı yazarken izleyeceğin 5 adım

- Prototipi tam doğru yaz.** Parametre isimleri ve tipleri (örneğin `const char *s`, `size_t n`) değerlendiricinin gözünde ilk izlenim -- prototipi ezberle bilmiyorsan bile mantıklı isimlerle (`dst/src/n` gibi `libft.h`'daki gibi) yaz, tutarlı ol.
- Önce değişkenlerini düşün, sonra döngüyü yaz.** Neredeyse her fonksiyon aynı iskelete oturuyor: bir/iki index değişkeni (`size_t i`), gerekiyorsa bir sonuç pointer'ı. Değişkenleri baştan tanımlarsan döngü yazarken kafan karışmaz.
- Önce NULL / sınır kontrolünü yaz, sonra ana mantığı.** Fonksiyonun en başında "bu girdi geçersizse ne olur" sorusunu cevapla (çoğu fonksiyonda bu tek satırlık bir `if`). Bunu unutursan hem norm hem de eval'de "eksik" görünür.
- Norm'u elinden geldiğince uygula ama saplanma.** Tab kullan, değişken bildirimlerinden sonra bir boş satır bırak. Küçük bir boşluk hatası seni batırmaz -- değerlendirici mantığı doğru mu diye bakıyor, norminette'i elle çalıştırmıyor bu aşamada.
- Yazdıktan sonra sesli olarak mantığını anlat.** "Burada üst üste binme varsa sondan başa kopyalıyorum çünkü..." gibi. Bu hem senin kontrol etmene yardım eder hem de değerlendiriciye "gerçekten anlıyorum" mesajı verir -- ki eval sheet'teki en kötü flag'lerden biri tam olarak "Can't support / explain code".

23 fonksiyonun hızlı hatırlatma tablosu

Bu tablo kod değil, **hatırlatma**. Amaç ezberlemek değil -- her satırı okuyup "evet, bunu neden öyle yazdığımı biliyorum" diyebiliyorsan hazırsın demektir. Bir satırda takılırsan, önceki dökümanına ([Libft_Rehberi.pdf](#)) dönüp o fonksiyonun tam anlatımını tekrar oku.

<code>ft_isalpha(int c)</code>	<code>'A'-'Z' 'a'-'z' -> 1/0</code>
<code>ft_isdigit(int c)</code>	<code>'0'-'9' -> 1/0</code>
<code>ft_isalnum(int c)</code>	<code>isalpha(c) isdigit(c)</code>
<code>ft_isascii(int c)</code>	<code>0 <= c <= 127</code>
<code>ft_isprint(int c)</code>	<code>32 <= c <= 126</code>
<code>ft_strlen(const char *s)</code>	<code>'\0'e kadar say</code>
<code>ft_memset(void *b, int c, size_t len)</code>	<code>unsigned char* cast, len kere *ptr++ = value</code>
<code>ft_bzero(void *s, size_t n)</code>	<code>ft_memset(s, 0, n)</code>
<code>ft_memcpy(void *dst, const void *src, n)</code>	<code>byte byte kopyala; dst&&src NULL ise NULL dön</code>
<code>ft_memmove(void *dest, const void *src, n)</code>	<code>d < s ise bastan; değilse *(d+n)=*(s+n) ile sondan</code>
<code>ft_strncpy(char *dst, const char *src, size)</code>	<code>size-1'e kadar kopyala, HER ZAMAN strlen(src) döndür</code>
<code>ft_strlcat(char *dst, const char *src, size)</code>	<code>dst_len bul; size<=dst_len ise erken cik; değilse ekle</code>
<code>ft_toupper(int c)</code>	<code>'a'-'z' ise c - 32</code>
<code>ft_tolower(int c)</code>	<code>'A'-'Z' ise c + 32</code>
<code>ft_strchr(const char *s, int c)</code>	<code>ilk eslesme; c=='\0' özel durumu unutma</code>
<code>ft_strrchr(const char *s, int c)</code>	<code>last degiskeniyle EN SON eslesmeyi tut</code>
<code>ft_strncmp(const char *s1, s2, size_t n)</code>	<code>(unsigned char) farki döndür; '\0' -> erken 0</code>
<code>ft_memchr(const void *s, int c, size_t n)</code>	<code>byte byte ara, bulunca pointer döndür</code>
<code>ft_memcmp(const void *s1, s2, size_t n)</code>	<code>(unsigned char) farki döndür</code>
<code>ft_strnstr(const char *big, little, len)</code>	<code>little[0]=='\0' özel durum; ic dongude j ile karsilastir</code>
<code>ft_atoi(const char *str)</code>	<code>bosluk atla -> +/- isaret -> rakamlari topla (long ile!)</code>
<code>ft_calloc(size_t nmemb, size_t size)</code>	<code>carpma tasmasi kontrolu -> malloc -> memset(0)</code>
<code>ft_strdup(const char *s)</code>	<code>strlen+1 malloc, karakter karakter kopyala, '\0' ekle</code>

Pratik yöntemi: 5 dakikalık turlar

En etkili hazırlık yöntemi pasif okuma değil, **aktif deneme**. Önerim:

1. Yukarıdaki listeden rastgele bir fonksiyon seç (gözünü kapatıp parmağını tabloya değdir gibi düşün).
2. Boş bir `.c` dosyasına, hiçbir şeye bakmadan, 5 dakika süre tutarak fonksiyonu baştan yaz.
3. Süre bitince gerçek dosyanla (`C:\42\42-Git-Projects\Libft\ft_XXX.c`) karşılaştır. Farkları not al -- genelde ya bir edge case unutulur (NULL kontrolü, size==0) ya da değişken ismi/tipi karışır.
4. Derleyip küçük bir `printf` ile gerçekten test et -- eval'de tam olarak bunu isteyecekler.
5. Bunu günde 3-4 fonksiyonla tekrarla, birkaç gün içinde 23'ün hepsini en az bir kez sıfırdan yazmış olursun.

EN ÇOK KARIŞTIRILAN NOKTALAR

Pratik yaparken özellikle şu üç noktaya dikkat et, çünkü canlı kodlamada en çok burada takılır: **(1)** `ft_strlcpy` / `ft_strlcat` 'in dönüş değerinin kırpma olsa bile her zaman "olması gereken" uzunluğu vermesi. **(2)** `ft_memmove` 'da hangi yönde kopyalanacağına `d < s` karşılaştırmasıyla karar verilmesi. **(3)** `ft_atoi` ve `ft_calloc` 'ta taşma (overflow) riskinden dolayı ara hesaplamanın `long` ile yapılması / kontrol edilmesi.

Puanlama Etiketleri (Flags) Ne Anlama Geliyor

Formun en altındaki etiketler değerlendiricinin sonucu nasıl işaretleyeceğini gösteriyor. Hangisinin ne anlama geldiğini ve -- elinde olanlar için -- nasıl kaçınacağını burada topladım.

Ok

Standart geçer not. Mandatory kısmın beklendiği gibi çalıştığı, sorun olmadığı anlamına gelir. Şu anki durumun (norm temiz, 61/61 test, derleme hatasız) bu notu hak ediyor.

Outstanding project

Beklenenin üzerinde bir iş çıkardığında verilir -- temiz kod, iyi test kapsamı, fonksiyonları rahatça açıklayabilme genelde bu izlenimi güçlendirir. Canlı kodlama sırasında sadece doğru yazmakla kalmayıp mantığını akıcı anlatabilmen bu yönde bir fark yaratabilir.

Empty Work

Proje boşsa ya da dosya/dizin adları yanlışsa kullanılır, not otomatik 0 olur. Senin projende dosya isimleri (ft_isalpha.c, Makefile, libft.h, README.md) doğru yapıda -- bu riski taşıyorsun, ama git repo'nun doğru dizine push edildiğinden emin ol.

Incomplete Work

Bazı fonksiyonlar eksik ya da çalışmıyor demek. Şu an 43 fonksiyonun 43'ü de derleniyor ve test ediliyor, bu riskin düşük olduğunu gösteriyor.

Invalid Compilation

-Wall -Wextra -Werror ile derleme başarısız olursa kullanılır. Az önce doğruladım: hem make hem make bonus hem de tüm dosyaların birlikte derlenmesi hatasız, uyarısız geçiyor.

Norme Error

Norminette hata verirse kullanılır, not otomatik 0 olur. Az önce 43 dosyanın hepsini norminette'ten geçirdim, hepsi "OK" -- şu an bu riske karşı temizsin, ama evoya girmeden hemen önce kendi terminalinde bir kez daha `norminette *.c *.h` çalıştır, olur da son anda bir düzenleme norm'u bozmuş olabilir.

Cheat

Hile şüphesi durumunda kullanılır, değerlendirme hemen durur. Kodunu gerçekten yazdığını, her satırın mantığını bildiğini gösterebiliyorsan (tam olarak bu dokümanın Bölüm 4'ünün hazırladığı şey) bu bir risk değil.

Crash

Program ya da testler sırasında segfault/çökme olursa kullanılır. Test suite'in 61/61 çökmeden geçiyor; yine de değerlendirici NULL, boş string, 0 gibi sınır değerlerle deneyebilir -- Bölüm 3'teki "ek uç durum" önerilerini gözden geçirmen bu riski daha da azaltır.

Concerning Situation

Net bir hata olmasa da değerlendiriciyi rahatsız eden bir durum (garip bir tasarım kararı, açıklayamadığın bir kod parçası vb.) için kullanılır. Kodun her satırının neden öyle yazıldığını bilmen -- bu dökümanın ve önceki Libft_Rehberi.pdf'in amacı -- bu riski minimuma indirir.

Leaks

Bellek sızıntısı bulunursa kullanılır. Bu ortamda valgrind çalıştıramadım; evoya girmeden önce mutlaka kendi terminalinde `valgrind --leak-check=full ./test_libft` ile kontrol et (Bölüm 2'de komut var).

Forbidden Function

İzin verilmeyen bir libc fonksiyonu (strcpy, memcpy'nin kendi implementasyonunun dışında kullanımı vb.) kullanıldığında işaretlenir. Kendi .c dosyalarında sadece malloc, free, write ve kendi ft_ fonksiyonlarının çağrıldığını doğruladım -- temiz.

Can't Support / Explain Code

Yazdığın (ya da canlı yeniden yazdığın) kodu açıklayamazsan kullanılır. Bu, hazırlığının en can alıcı kısmı -- Bölüm 4'teki pratik turlarını gerçekten yaparsan bu riskten en çok kaçınabileceğin flag bu.

Evo Günü: Son Kontrol Listesi

Evoya girmeden hemen önce, sırayla:

- ☐ `git status` temiz mi, `git push` yapıldı mı -- değerlendirici klonlayacağı için lokal değişiklikler yetmez.
- ☐ Temiz bir klasöre `git clone` ile projeyi çek, orada `make` ve `make bonus` çalıştır -- "benim makinemde çalışıyordu" tuzağına düşme.
- ☐ `norminette *.c *.h` son bir kez çalıştır, hepsi "OK" olmalı.
- ☐ `gcc -Wall -Wextra -Werror *.c -o test_libft && ./test_libft` ile 61/61'i son bir kez gözünle gör.
- ☐ `valgrind --leak-check=full ./test_libft` ile sızıntı olmadığını doğrula.
- ☐ README.md'nin ilk satırını bir kez daha oku, tam istenen italik formatta olduğundan emin ol.
- ☐ Bölüm 4'teki 23 satırlık hatırlatma tablosuna son bir kez göz gezdir -- özellikle son 2-3 gün içinde **sıfırdan yazmadığın** fonksiyonlara odaklan.
- ☐ Derin bir nefes al. Bu projeyi baştan sona sen yazdın, hatalarını sen bulup düzelttin (bir kısmında yardım aldın, README'de bunu zaten dürüstçe belirtmişsin) -- kodun her satırının neden orada olduğunu biliyorsun.

Hazırsın

Norminette temiz, derleme hatasız, 61 test yeşil, global değişken yok, yasak fonksiyon yok, README eksiksiz. Formdaki dört preliminary test'i de geçiyorsun. Geriye kalan tek şey canlı kodlama -- ve onun için elindeki en iyi araç, bu 23 fonksiyonu birkaç kez daha sıfırdan yazmış olmak. Bol şans!